

The Composition Problem *

J. J. Liu¹, Y. L. Wang^{2,†}, C. J. Hsu², and S. F. Yu²

¹ Department of Information Management,
Shih Hsin University, Taipei, Taiwan
jjliu@cc.shu.edu.tw

² Department of Information Management,
National Taiwan University of Science and Technology, Taipei, Taiwan
ylwang@cs.ntust.edu.tw, chengju.hsu@gmail.com, rogeryu@mail.ntust.edu.tw

Abstract

In this paper, we define a new problem called the composition problem which can be viewed as a parallel version of the scheduling problem. The composition problem is to find two disjoint subsets S_a and S_b from $S = \{(a_i, b_i) | 1 \leq i \leq n\}$ with $|S_a| + \frac{|S_b|}{2} = k$ such that $\sum_{s_i \in S_a} a_i + \sum_{s_i \in S_b} b_i$ is minimum where k is a given integer and both a_i and b_i are real numbers with $a_i > b_i$. We shall propose an efficient algorithm for solving this problem.

Keywords: Greedy method, scheduling problem, optimization.

1 The Composition Problem

In this paper, we shall discuss a new problem which can be viewed as a parallel version of the scheduling problem. An application of this problem is described as follows. Assume that there are n different available processors. We want to rent k processors from them and execute simultaneously to finish our work. For each processor, we can use it alone or sharing with others. For convenience, we call them *alone* and *sharing*, respectively, processors. Clearly, each alone processor is much expensive than its sharing processor. Moreover, each alone processor can be replaced by 2 different shar-

ing processors if the total fee is cheaper than the alone processor.

The problem is described formally as follows. Let S be a set of n integer pairs $s_i = (a_i, b_i)$, $i = 1, 2, \dots, n$, with $a_i > b_i$ and k is an integer. Two sets S_a and S_b constitute a k -composition of S if sets S_a and S_b are two disjoint subsets of S such that $|S_a| + \frac{|S_b|}{2} = k$. The *composition value* with respect to S_a and S_b , denoted by $\lambda_k(S_a, S_b)$, is defined as $\sum_{s_i \in S_a} a_i + \sum_{s_i \in S_b} b_i$. The *composition problem* is to find two disjoint subsets S_a and S_b from S such that $\lambda_k(S_a, S_b)$ is minimum; and, in this case, S_a and S_b are denoted by S_a^* and S_b^* , respectively.

For instance, let

$$S = \left\{ \begin{array}{cccccccccc} s_1 & s_2 & s_3 & s_4 & s_5 & s_6 & s_7 & s_8 & s_9 & s_{10} \\ 9 & 13 & 19 & 8 & 18 & 11 & 21 & 10 & 21 & 17 \\ 4 & 6 & 8 & 2 & 14 & 9 & 10 & 9 & 6 & 12 \end{array} \right\}$$

where $s_1 = (9, 4)$, $s_2 = (13, 6)$, ..., and $s_{10} = (17, 12)$. If $k = 3$, then $S_a = \emptyset$ and $S_b = \{s_1, s_2, s_3, s_4, s_8, s_9\}$ is a 3-composition of S . Thus, $\lambda_3(S_a, S_b) = \sum_{s_i \in S_a} a_i + \sum_{s_i \in S_b} b_i = 4 + 6 + 8 + 2 + 9 + 6 = 35$. Furthermore, $S_a^* = \{s_6, s_8\}$ and $S_b^* = \{s_1, s_4\}$ is a minimum 3-composition of S and $\lambda_3(S_a^*, S_b^*) = 11 + 10 + 4 + 2 = 27$.

The remaining part of this paper is organized as follows. In Section 2, we propose a greedy algorithm for solving the composition problem. In Section 3, we prove the correctness of our algorithm. In Section 4, we describe an efficient way to solve the critical step in our algorithm. Finally, our concluding remarks will be given in the last section.

*This work was supported in part by the National Science Council of the Republic of China under contracts NSC 97-2218-E-128-001- and NSC 97-2221-E-011-158-MY3.

†All correspondence should be addressed to Professor Yue-Li Wang, Department of Information Management, National Taiwan University of Science and Technology, Taipei, Taiwan, ROC (Email: ylwang@cs.ntust.edu.tw).

2 An Algorithm for Solving the Composition Problem

For an element $s_i = (a_i, b_i)$, $1 \leq i \leq n$ in S , we call a_i (respectively, b_i) the a -value (respectively, b -value) of s_i and $c_i = a_i - b_i$ the c -value of s_i . For simplicity, we assume that, for any distinct elements s_i and s_j in S , at least one of their a -values or b -values must be different. Let $Max(T, x, j)$ (respectively, $Min(T, x, j)$) stand for the pair, say s_i , whose x -value is the j -th largest (respectively, j -th smallest) x -value in set T where $x \in \{a, b, c\}$. Note that if two elements have the same a -value (respectively, b -value), then the one with a smaller b -value (respectively, a -value) has a smaller rank. Proposition 1 reveals that only $O(k)$ elements of S will be considered in finding S_a^* and S_b^* .

Proposition 1 $S_a^* \subseteq \bigcup_{1 \leq j \leq 2k-1} Min(S, a, j)$ and $S_b^* \subseteq \bigcup_{1 \leq j \leq 2k} Min(S, b, j)$.

Let $s_{\beta_1} = Max(S_b, b, 1)$ and $s_{\beta_2} = Max(S_b, b, 2)$. For convenience, we also use a_{β_1} and a_{β_2} to denote the a -values of s_{β_1} and s_{β_2} , respectively. Similarly, the same convention is also applied on their b -values and other named pairs. The element $Min(S_r, a, 1)$ is denoted by s_δ , where $S_r = S \setminus (S_a \cup S_b)$. We say that there exists an S_r -improvement on s_δ with respect to S_a and S_b if $a_\delta < b_{\beta_1} + b_{\beta_2}$ and, in this case, s_δ is called the S_r -candidate. The reason is that the k -composition $S'_a = S_a \cup \{s_\delta\}$ and $S'_b = S_b \setminus \{s_{\beta_1}, s_{\beta_2}\}$ has

$$\begin{aligned} \lambda_k(S'_a, S'_b) &= \lambda_k(S_a, S_b) + a_\delta - (b_{\beta_1} + b_{\beta_2}) \\ &< \lambda_k(S_a, S_b), \end{aligned}$$

where “ $\setminus$ ” is the difference operation of two sets. For simplicity, in the following, we shall use $S - s_x$ (respectively, $S + s_x$) to replace $S \setminus \{s_x\}$ (respectively, $S \cup \{s_x\}$) when the second operand is a one-element set in the difference (respectively, union) operation. Similarly, there is an S_b -improvement on $s_i \in S_b$ with respect to S_a and S_b if $a_i < b_i + b_{\beta_1}$ when $s_i \neq s_{\beta_1}$ and $a_i < b_i + b_{\beta_2}$ for $s_i = s_{\beta_1}$ and, in this case, s_i is called an S_b -candidate with respect to S_a and S_b . In an S_b -improvement on $s_i \in S_b$ with respect to S_a and S_b , we can also find a better k -composition as $S'_a = S_a + s_i$ and $S'_b = S_b \setminus \{s_i, s_{\beta_1}\}$ for the former case and $S'_a = S_a + s_i$ and $S'_b = S_b \setminus \{s_i, s_{\beta_2}\}$ for the latter case with $\lambda_k(S'_a, S'_b) < \lambda_k(S_a, S_b)$. For brevity, we shall not mention “with respect to S_a and S_b ” again when talking about candidates if it will not make confusion. An element s_x is called a *candidate* of S if it is either an S_b -candidate or an S_r -candidate.

An S_b -candidate $s_x \in S_b$ is called an S_b -forbidden element if there exist another two distinct s_i and $s_j \in S_b$ and none of them equal to $Max(S_b - s_x, b, 1)$ such that $max\{a_i, a_j\} < a_x$ and $c_i + c_j < a_x$. An S_r -candidate s_δ is called an S_r -forbidden element if there exist two distinct elements $s_i, s_j \in S_b \setminus \{s_{\beta_1}, s_{\beta_2}\}$ such that $max\{a_i, a_j\} < a_x$ and $c_i + c_j < a_x$. An element s_x is called a *forbidden element* of S if it is either an S_b -forbidden element or an S_r -forbidden element. The two corresponding elements s_i and s_j of a forbidden element s_x are called the *forcing pair* of s_x . The meaning behind a forbidden element is that if we put a forbidden element s_x into S_a , then, in the long run, s_x will be removed from S_a and replaced by its forcing pair s_i and s_j to get a better composition. That is, $\lambda_k(S_a, S_b) > \lambda_k((S_a \cup \{s_i, s_j\}) - s_x, S_b \setminus \{s_i, s_j\})$ if s_x was already put into S_a . Since we shall use the greedy method to solve this problem, including a forbidden element before one of its forcing pair will violate the principle of the greedy method, That is, an element is not removed from a solution set any more after it was already put into the solution set [2].

To easily description later, we use a one-element set S_{rc} to denote the set containing the nonforbidden S_r -candidate if it exists. Meanwhile, the set which contains all nonforbidden S_b -candidates is denoted by S_{bc} . In the previous example, we can find that $s_{\beta_1} = s_8$, $s_{\beta_2} = s_3$, and $s_\delta = s_6$. Element s_δ is an S_r -candidate since $b_{\beta_1} + b_{\beta_2} = 9 + 8 > 11 = a_\delta$. Moreover, no $s_i, s_j \in S_b$ exist such that s_δ becomes a forbidden element. Hence, $S_{rc} = \{s_6\}$. We can also find that s_1, s_2, s_4 , and s_8 are S_b -candidates while s_3 and s_9 are not. Among them, only s_2 is a forbidden element since s_1 and s_4 are its forcing pair. That is, $max\{a_1, a_4\} < a_2$ and $c_1 + c_4 = 5 + 6 < 13 = a_2$. Therefore, $S_{bc} = \{s_1, s_4, s_8\}$.

Lemma 2 *If there is a forbidden element with respect to S_a and S_b , then there exists a nonforbidden candidate.*

Proof. Let S_c be the candidate set containing all S_b -candidates and the S_r -candidate. If there is a forbidden element, then $|S_c| \geq 1$. Let $s_x = Min(S_c, a, 1)$. In the following, we shall prove that s_x cannot be a forbidden element.

Assume on the contrary that s_x is a forbidden element. Then it is either an S_b -candidate or the S_r -candidate. We only prove the first case, i.e., $s_x \in S_b$. The other case can be handled similarly. By the definition of S_b -forbidden elements, there exist another two distinct elements $s_i, s_j \in S_b$ and none of them equal to $Max(S_b \setminus \{s_x\}, b, 1)$

such that $\max\{a_i, a_j\} < a_x$ and $c_i + c_j < a_x$. Let $s_k = \text{Max}(S_b \setminus \{s_x\}, b, 1)$. Since s_x is an S_b -candidate, $a_x < b_x + b_k$. Thus,

$$\begin{aligned} c_i + c_j &< a_x \\ \Rightarrow (a_i - b_i) + (a_j - b_j) &< b_x + b_k \\ \Rightarrow a_i + a_j &< b_i + b_j + b_x + b_k \\ \Rightarrow a_i + a_j &< b_i + b_j + 2b_{\beta_1}. \end{aligned} \quad (1)$$

By Equation (1), it is impossible that both $a_i \geq b_i + b_{\beta_1}$ and $a_j \geq b_j + b_{\beta_1}$. Therefore, at least one of s_i and s_j , say s_i , is an S_b -candidate. It contradicts that s_x is the S_b -candidate with the smallest a -value among all elements is S_c . This completes the proof. $\square$

The following notation will be used to determine which element in $S_{bc} \cup S_{rc}$ will be selected to do an S_r -improvement or an S_b -improvement when S_b is not empty.

$$\begin{aligned} \Delta_r &= \begin{cases} (b_{\beta_1} + b_{\beta_2}) - a_\delta & \text{if } S_{rc} \neq \emptyset, \\ -\infty & \text{otherwise,} \end{cases} \\ \Delta_{b_i} &= \begin{cases} (b_{\beta_1} + b_{\gamma_i}) - a_{\gamma_i} & \text{if } s_{\gamma_i} \neq s_{\beta_1}, \\ (b_{\beta_1} + b_{\beta_2}) - a_{\gamma_i} & \text{if } s_{\gamma_i} = s_{\beta_1}, \end{cases} \\ \Delta_b &= \begin{cases} \text{Max}\{\Delta_{b_1}, \Delta_{b_2}\}, & \text{if } S_{bc} \neq \emptyset, \\ -\infty & \text{otherwise,} \end{cases} \end{aligned}$$

where $s_{\gamma_i} = \text{Min}(S_{bc}, c, i)$. Note that if two elements have the same c -value, then the one with a smaller a -value has a smaller rank. From the formula for computing Δ_r and comparing it with the definition of an S_r -improvement, we can see that Δ_r is the amount of improving value from $\lambda_k(S_a, S_b)$ if an S_r -improvement is done on s_δ . Similarly, Δ_{b_i} is the amount of improving value from $\lambda_k(S_a, S_b)$ if an S_b -improvement is done on s_{γ_i} .

Lemma 3 $\max_{1 \leq i \leq |S_{bc}|} \{\Delta_{b_i}\} = \max\{\Delta_{b_1}, \Delta_{b_2}\}$.

Proof.

First, we consider the case where $s_{\gamma_1} = s_{\beta_1}$. Then, by definition, $\Delta_{b_1} = (b_{\beta_1} + b_{\beta_2}) - a_{\beta_1}$. Furthermore, $\Delta_{b_2} = (b_{\beta_1} + b_{\gamma_2}) - a_{\gamma_2} \geq (b_{\beta_1} + b_{\gamma_i}) - a_{\gamma_i} = \Delta_{b_i}$ for $i \geq 3$. Thus, in this case, $\max_{1 \leq i \leq |S_{bc}|} \{\Delta_{b_i}\} = \max\{\Delta_{b_1}, \Delta_{b_2}\}$. For the case where $s_{\gamma_1} \neq s_{\beta_1}$, we can find that $\Delta_{b_1} = (b_{\beta_1} + b_{\gamma_1}) - a_{\gamma_1} \geq (b_{\beta_1} + b_{\gamma_i}) - a_{\gamma_i} = \Delta_{b_i}$ for $i \geq 2$ and $s_{\gamma_i} \neq s_{\beta_1}$. Assume that $s_{\gamma_i} = s_{\beta_1}$ for some $i \neq 1$. Then,

$$\begin{aligned} \Delta_{b_1} &= b_{\beta_1} + b_{\gamma_1} - a_{\gamma_1} \\ &\geq b_{\beta_1} + b_{\gamma_i} - a_{\gamma_i} \\ &= b_{\beta_1} + b_{\beta_1} - a_{\gamma_i} \\ &\geq b_{\beta_2} + b_{\beta_1} - a_{\gamma_i} \\ &= \Delta_{b_i}. \end{aligned}$$

This means that, in this case, $\max_{1 \leq i \leq |S_{bc}|} \{\Delta_{b_i}\} = \Delta_{b_1}$ and the lemma follows. $\square$

Now, we are at a position to describe our greedy algorithm.

Algorithm 1

Input: A set S of n integer pairs (a_i, b_i) , for $i = 1, 2, \dots, n$, and an integer k .

Output: Two disjoint subsets S_a and S_b of S .

begin

Step 1. Let $S_a = \emptyset$, $S_b = \bigcup_{1 \leq i \leq 2k} \{\text{Min}(S, b, i)\}$, and $S_r = S \setminus (S_a \cup S_b)$.

Step 2. Find S_{rc} and S_{bc} from S_r and S_b , respectively. If both sets are empty, then output S_a and S_b and terminate.

Step 3. Compare Δ_r and Δ_b to determine which of the following cases will be executed. Assume that $\Delta_b = \Delta_{b_j}$ for some $j = 1$ or 2 .

Case 1. If $\Delta_r \geq \Delta_b$, then make an S_r -improvement on s_δ and update related sets, i.e., $S_a = S_a + s_\delta$, $S_b = S_b \setminus \{s_{\beta_1}, s_{\beta_2}\}$, and $S_r = (S_r \cup \{s_{\beta_1}, s_{\beta_2}\}) - s_\delta$.

Case 2. If $\Delta_r < \Delta_b$ and $s_{\gamma_j} \neq s_{\beta_1}$, then make an S_b -improvement on s_{γ_j} and update related sets, i.e., $S_a = S_a + s_{\gamma_j}$, $S_b = S_b \setminus \{s_{\beta_1}, s_{\gamma_j}\}$, and $S_r = S_r + s_{\beta_1}$.

Case 3. If $\Delta_r < \Delta_b$ and $s_{\gamma_j} = s_{\beta_1}$, then make an S_b -improvement on s_{γ_j} and update related sets, i.e., $S_a = S_a + s_{\gamma_j}$, $S_b = S_b \setminus \{s_{\beta_2}, s_{\gamma_j}\}$, and $S_r = S_r + s_{\beta_2}$.

Step 4. Go to Step 2.

end

We also use the previous example to illustrate **Algorithm 1**. All the changes of S_a , S_b , S_r , S_{bc} , S_{rc} , Δ_r , Δ_b , and the improving activity in each iteration are shown in Table 1. Note that the above sets and values are computed before Step 3 of **Algorithm 1**.

Table 1: The changes in each iteration

Iteration	S_a	S_b	S_{bc}	S_{rc}
1	$\emptyset$	$s_1, s_2, s_3, s_4, s_8, s_9$	s_1, s_4, s_8	s_6
2	s_8	s_1, s_2, s_4, s_9	s_1	s_6
3	s_6, s_8	s_1, s_4	$\emptyset$	$\emptyset$
Δ_r	Δ_b	$\lambda_3(S_a, S_b)$		
6	7	35		
1	1	28		
—	—	27		

3 The Correctness of Algorithm 1

For proving $\lambda_k(S_a, S_b)$ is minimum, we assign each pair s_i a label in **Algorithm 1**, where S_a and S_b are the output of **Algorithm 1**. Thus, we rewrite **Algorithm 1** as **Algorithm 2** in which the labeling system is included.

Algorithm 2

Input: A set S of n integer pairs (a_i, b_i) , for $i = 1, 2, \dots, n$, and an integer k .

Output: Two disjoint subsets S_a and S_b of S .
begin

Step 1. Let $S_a = \emptyset$, $S_b = \bigcup_{1 \leq i \leq 2k} \{ \text{Min}(S, b, i) \}$,
 $S_r = S \setminus (S_a \cup S_b)$, and $i = 1$.

Step 2. Find S_{rc} and S_{bc} from S_r and S_b , respectively.

Substep 2.1. If $s_\alpha \notin S_{bc}$ is not labeled and $a_\alpha < a_\delta$, then $\ell_\alpha = i$ and $i = i + 1$ where $s_\alpha = \text{Min}(S_b, a, 1)$ and ℓ_x denotes the label assigned to s_x .

Substep 2.2. If both S_{rc} and S_{bc} are empty, then label all unlabeled elements from i to n according to their a -value in increasing order and output S_a and S_b before terminate.

Step 3. Compare Δ_r and Δ_b to determine which of the following cases will be executed.

Assume that $\Delta_b = \Delta_{b_j}$ for some $j = 1$ or 2 .

Case 1. If $\Delta_r \geq \Delta_b$, then make an S_r -improvement on s_δ and update related sets, i.e.,
 $S_a = S_a + s_\delta$, $S_b = S_b \setminus \{s_{\beta_1}, s_{\beta_2}\}$, and
 $S_r = (S_r \cup \{s_{\beta_1}, s_{\beta_2}\}) - s_\delta$.

Case 2. If $\Delta_r < \Delta_b$ and $s_{\gamma_j} \neq s_{\beta_1}$, then make an S_b -improvement on s_{γ_j} and update related sets, i.e.,
 $S_a = S_a + s_{\gamma_j}$, $S_b = S_b \setminus \{s_{\beta_1}, s_{\gamma_j}\}$, and
 $S_r = S_r + s_{\beta_1}$.

Case 3. If $\Delta_r < \Delta_b$ and $s_{\gamma_j} = s_{\beta_1}$, then make an S_b -improvement on s_{γ_j} and update related sets, i.e.,
 $S_a = S_a + s_{\gamma_j}$, $S_b = S_b \setminus \{s_{\beta_2}, s_{\gamma_j}\}$, and
 $S_r = S_r + s_{\beta_2}$.

Step 4. Label the selected element, say s_x , in Step 3, i.e., $\ell_x = i$ and set $i = i + 1$.
Go to Step 2.

end

From an observation on the labeling system in **Algorithm 2**, there are three places, namely Substep 2.1, Substep 2.2 and Step 4, at which a label is assigned to an element of S . For convenience,

the labels obtained from Substep 2.1, Substep 2.2 and Step 4 are called *skipping labels*, *ending labels*, and *improving labels*, respectively. The iteration in **Algorithm 2** at which both S_{rc} and S_{bc} are empty is called the *ending iteration*. To distinguish the final sets obtained in **Algorithm 2**, we use S_a^t, S_b^t, S_{bc}^t , and S_{rc}^t to denote temporary sets S_a, S_b, S_{bc} , and S_{rc} , respectively, found at iteration t of **Algorithm 2**. Accordingly, let $s_{\beta_1}^t = \text{Max}(S_b^t, b, 1)$, $s_{\beta_2}^t = \text{Max}(S_b^t, b, 2)$, etc.

Now we use the example in Section 1 to illustrate **Algorithm 2**. After **Algorithm 2** is terminated, the labels of all elements are $\ell_8 = 1$, $\ell_4 = 2$, $\ell_6 = 3$, $\ell_1 = 4$, $\ell_2 = 5$, $\ell_{10} = 6$, $\ell_5 = 7$, $\ell_3 = 8$, $\ell_9 = 9$, and $\ell_7 = 10$. The reason for $\ell_8 = 1$ is that s_8 is the first selected element from $S_{bc}^1 \cup S_{rc}^1$ to do an S_b -improvement and ℓ_8 is an improving label. The reason for $\ell_4 = 2$ is that s_4 meets the requirement described in Substep 2.1 in the second iteration of **Algorithm 2**. That is, $s_4 = \text{Min}(S_b^2, a, 1)$ and $s_4 \notin S_{bc}^2$. Furthermore, s_4 is not labeled and $a_4 < a_6$ where s_6 is the s_δ^2 of S_r^2 . Thus, ℓ_4 is a skipping label. The reason for $\ell_6 = 3$ is that s_6 is the selected element from $S_{bc}^2 \cup S_{rc}^2$ to do an S_b -improvement in the second iteration and ℓ_6 is also an improving label. The labels of the remaining elements are labeled in Substep 2.2 since both S_{rc}^3 and S_{bc}^3 are empty. Therefore, all of them are ending labels. Note that the ending iteration of **Algorithm 2** is iteration 3.

In the following, we shall prove that $\lambda_k(S_a, S_b) = \lambda_k(S_a^*, S_b^*)$ where S_a and S_b are the output of **Algorithm 2**. We assume without loss of generality that $S_a = (s_{p_1}, s_{p_2}, \dots, s_{p_{n'}})$ and $S_b = (s_{q_1}, s_{q_2}, \dots, s_{q_{m'}})$ are the output of **Algorithm 2**, where $\ell_{p_1} < \ell_{p_2} < \dots < \ell_{p_{n'}}$. The pair $S_a^* = (s_{x_1}, s_{x_2}, \dots, s_{x_{n^*}})$ and $S_b^* = (s_{y_1}, s_{y_2}, \dots, s_{y_{m^*}})$ used in our proof are also labeled by the same labels found in **Algorithm 2**, i.e., $\ell_{x_1} < \ell_{x_2} < \dots < \ell_{x_{n^*}}$. Furthermore, if there are more than one optimal k -composition, then we choose the one with the smallest lexicographic order by labels of S_a^* among all optimal k -compositions of minimum size $|S_a^*|$.

Proposition 4 *If $\lambda_k(S_a, S_b)$ is minimum, then S_b can be changed to contain all elements of $\text{Min}(S \setminus S_a, b, i)$ for $i = 1, 2, \dots, |S_b|$.*

By Proposition 4, in the remaining part of this paper, we assume that any S_b contains all elements of $\text{Min}(S \setminus S_a, b, i)$ for $i = 1, 2, \dots, |S_b|$.

Lemma 5 *If s_{x_i} and s_{p_i} , $1 \leq i \leq \min\{n^*, n'\}$, are the first different elements between S_a^* and S_a*

with $\ell_{x_i} > \ell_{p_i}$ and $s_{p_i} \notin S_b^*$, then $a_{x_j} < a_{p_i}$ for $j = i, i+1, \dots, n^*$.

Proof. We assume on the contrary that there exists an element s_{x_j} in S_a^* for some j , $i \leq j \leq n^*$, such that $a_{x_j} \geq a_{p_i}$. Obviously, in this case, $s_{p_i} \notin S_a^*$. Thus, joining s_{p_i} into S_a^* and removing s_{x_j} from S_a^* , we can obtain a new k -composition $S'_a = S_a^* + s_{p_i} - s_{x_j}$ and $S'_b = S_b^*$. Note that since $s_{p_i} \notin S_b^*$, the elements in S'_b remain unchanged. This yields

$$\begin{aligned} & \lambda_k(S'_a, S'_b) \\ &= \lambda_k(S_a^* + s_{p_i} - s_{x_j}, S_b^*) \\ &= \lambda_k(S_a^*, S_b^*) + a_{p_i} - a_{x_j} \\ &\leq \lambda_k(S_a^*, S_b^*). \end{aligned}$$

In the above derivation, if $a_{x_j} > a_{p_i}$, then $\lambda_k(S_a^*, S_b^*) > \lambda_k(S'_a, S'_b)$. It contradicts that $\lambda_k(S_a^*, S_b^*)$ is minimum. If $a_{x_j} = a_{p_i}$, then $\lambda_k(S_a^*, S_b^*) = \lambda_k(S'_a, S'_b)$. The lexicographic order of S'_a will be smaller than S_a^* . It also contradicts our assumption. We conclude that $a_{x_j} < a_{p_i}$ for every $j = i, i+1, \dots, n^*$. $\square$

Lemma 6 If s_{x_i} and s_{p_i} , $1 \leq i \leq \min\{n^*, n'\}$, are the first different elements between S_a^* and S_a with $\ell_{x_i} > \ell_{p_i}$ and $s_{p_i} \notin S_b^*$, then every s_{x_j} is in S_b^t for $j = i, i+1, \dots, n^*$ where iteration t is the iteration with $S_a^t = \{s_{p_1}, s_{p_2}, \dots, s_{p_{i-1}}\}$.

Proof. Assume on the contrary that there exists an $s_{x_j} \in S_r^t$, $i \leq j \leq n^*$. By Lemma 5, $a_{x_j} < a_{p_i}$ for $j = i, i+1, \dots, n^*$. Since $a_\delta^t \leq a_{x_j} < a_{p_i}$, ℓ_{p_i} cannot be either a skipping label or an ending label. This implies that $s_{p_i} \in S_b^t$. Furthermore, ℓ_{p_i} is assigned at iteration t of Algorithm 2 and $\Delta_b^t > \Delta_r^t$. However, $\Delta_r^t = b_{\beta_1}^t + b_{\beta_2}^t - a_\delta^t \geq b_{\beta_1}^t + b_{\beta_2}^t - a_{x_j} > b_{\beta_1}^t + b_{\beta_2}^t - a_{p_i} \geq \Delta_b^t$. It is a contradiction and the lemma follows. $\square$

Lemma 7 If s_{x_i} and s_{p_i} , $1 \leq i \leq \min\{n^*, n'\}$, are the first different elements between S_a^* and S_a with $\ell_{x_i} > \ell_{p_i}$ and $s_{p_i} \notin S_b^*$, then $s_{x_j} \neq s_{\beta_1}^t$ for $j = i, i+1, \dots, n^*$ where iteration t is the iteration with $S_a^t = \{s_{p_1}, s_{p_2}, \dots, s_{p_{i-1}}\}$.

Proof. By Lemmas 5 and 6, $a_{x_j} < a_{p_i}$ and s_{x_j} is in S_b^t for $j = i, i+1, \dots, n^*$. For the purpose of contradiction, we assume that $s_{x_j} = s_{\beta_1}^t$ for some j , $i \leq j \leq n^*$. Now we consider the following two cases.

Case 1: $s_{p_i} \notin S_b^t$.

If $s_{p_i} \notin S_b^t$, then $s_{p_i} = s_\delta^t$. The reason is that ℓ_{p_i} is assigned at iteration t of Algorithm 2, namely,

s_{p_i} is the improving element selected at iteration t . Hence, $\Delta_r^t = b_{\beta_1}^t + b_{\beta_2}^t - a_{p_i} \geq \Delta_b^t$. However, in this case, $\Delta_b^t \geq b_{\beta_1}^t + b_{\beta_2}^t - a_{x_j} > b_{\beta_1}^t + b_{\beta_2}^t - a_{p_i} = \Delta_r^t$. It is a contradiction.

Case 2: $s_{p_i} \in S_b^t$.

In this case, since s_{p_i} is an improving element selected at iteration t of Algorithm 2, $\Delta_b^t = b_{\beta_1}^t + b_{p_i} - a_{p_i} \geq b_{\beta_1}^t + b_{\beta_2}^t - a_{x_j}$ which implies $b_{\beta_2}^t - a_{p_i} \geq b_{p_i} - a_{p_i} \geq b_{\beta_2}^t - a_{x_j}$ and $a_{x_j} \geq a_{p_i}$. It is also a contradiction. This completes the proof. $\square$

Lemma 8 For $1 \leq i \leq \min\{n^*, n'\}$, $\ell_{x_i} = \ell_{p_i}$.

Proof. We assume for the purpose of contradiction that $\ell_{x_1} = \ell_{p_1}, \ell_{x_2} = \ell_{p_2}, \dots, \ell_{x_{i-1}} = \ell_{p_{i-1}}$, $\ell_{x_i} \neq \ell_{p_i}$ for some $i \leq \min\{n^*, n'\}$, i.e., s_{x_i} and s_{p_i} are the first different elements between S_a^* and S_a . Let $S_a^t = \{s_{p_1}, s_{p_2}, \dots, s_{p_{i-1}}\}$ and $S_b^t = \bigcup_{1 \leq i \leq 2(k-i+1)} \{Min(S \setminus S_a^t, b, i)\}$. Note that S_a^t and S_b^t are the two temporary sets S_a and S_b , respectively, in iteration t of Algorithm 2 when processing s_{p_i} . We have four cases to consider.

Case 1: $\ell_{x_i} < \ell_{p_i}$.

It is clear that ℓ_{x_i} must be a skipping label and $s_{x_i} = Min(S_b^t, a, 1)$ which is in $S_b^t \setminus S_{bc}^t$. This implies that $a_{x_i} \geq b_{\beta_1}^t + b_{\beta_2}^t$ if $s_{x_i} = s_{\beta_1}^t$ or $a_{x_i} \geq b_{x_i} + b_{\beta_1}^t$ if $s_{x_i} \neq s_{\beta_1}^t$. Let S'_a and S'_b constitute a $2/k$ -composition of S where $S'_a = S_a^* - s_{x_i}$. We can also find that $|S'_b| = m^* + 2$. Assume that $S'_b = S_b^* \cup \{s_{e_1}, s_{e_2}\}$. Let $s'_{m^*+1} = s_{e_1} = Max(S'_b, b, 1)$ and $s'_{m^*+2} = s_{e_2} = Max(S'_b, b, 2)$. Since $S'_b \subset S_b^t$ and $s_{\beta_j}^t = Max(S_b^t, b, j)$ for $j = 1$ or 2 , $b'_{m^*+1} \leq b_{\beta_1}^t$ and $b'_{m^*+2} \leq b_{\beta_2}^t$ where b'_{m^*+1} and b'_{m^*+2} are the b -values of s'_{m^*+1} and s'_{m^*+2} , respectively. If $s_{x_i} = s_{\beta_1}^t$, then $a_{x_i} \geq b_{\beta_1}^t + b_{\beta_2}^t \geq b'_{m^*+1} + b'_{m^*+2}$; otherwise, $a_{x_i} \geq b_{x_i} + b_{\beta_1}^t \geq b_{x_i} + b'_{m^*+1}$. Thus, in the former case,

$$\begin{aligned} & \lambda_{2,k}(S'_a, S'_b) \\ &= \lambda_{2,k}(S_a^* - s_{x_i}, S_b^* \cup \{s'_{m^*+1}, s'_{m^*+2}\}) \\ &\leq \lambda_{2,k}(S_a^*, S_b^*) - a_{x_i} + b'_{m^*+1} + b'_{m^*+2} \\ &\leq \lambda_{2,k}(S_a^*, S_b^*). \end{aligned}$$

Similarly, $\lambda_{2,k}(S'_a, S'_b) \leq \lambda_{2,k}(S_a^* - s_{x_i}, S_b^* \cup \{s_{x_i}, s'_{m^*+1}\}) \leq \lambda_{2,k}(S_a^*, S_b^*)$ for the latter case. In either case, $|S_a^*|$ is not minimum among all optimum solutions. This contradicts our assumption that $|S_a^*|$ is minimum. Therefore, this case is impossible.

Case 2: $\ell_{x_i} > \ell_{p_i}$ and $s_{p_i} \notin S_b^t$.

Since $S_b^t \subset S_b^*$ and $s_{p_i} \notin S_b^t$, $s_{p_i} \notin S_b^*$. By Lemmas 5, 6, and 7, $a_{x_j} < a_{p_i}$, $s_{x_j} \in S_b^t$, and $s_{x_j} \neq s_{\beta_1}^t$ for $j = i, i+1, \dots, n^*$. Since ℓ_{p_i} is the improving label assigned at iteration t of Algorithm 2, we

can find that $s_{p_i} = s_{\delta}^t$, $\Delta_r^t = b_{\beta_1}^t + b_{\beta_2}^t - a_{p_i}$, and $\Delta_r^t \geq \Delta_b^t$. If there exists an element $s_{x_j} = s_{\beta_2}^t$ for some j , $i \leq j \leq n^*$, then $\Delta_b^t \geq b_{\beta_1}^t + b_{\beta_2}^t - a_{x_j} > b_{\beta_1}^t + b_{\beta_2}^t - a_{p_i} = \Delta_r^t$. It is a contradiction. Therefore, s_{x_j} is neither $s_{\beta_1}^t$ nor $s_{\beta_2}^t$ for $j = i, i+1, \dots, n^*$.

Since $s_{p_i} \notin S_b^t$ and ℓ_{p_i} is an improving label, s_{p_i} is in S_{rc}^t at iteration t of Algorithm 2. This means that, for every $s_x, s_y \in S_b^t \setminus \{s_{\beta_1}^t, s_{\beta_2}^t\}$, if $\max\{a_x, a_y\} < a_{p_i}$, then $c_x + c_y \geq a_{p_i}$. Consequently, in this case, we can have the following derivation when $n^* > i$.

$$\begin{aligned} & \lambda_k((S_a^* + s_{p_i}) \setminus \{s_{x_i}, s_{x_{i+1}}\}, S_b^* \cup \{s_{x_i}, s_{x_{i+1}}\}) \\ &= \lambda_k(S_a^*, S_b^*) + a_{p_i} - a_{x_i} - a_{x_{i+1}} + b_{x_i} + b_{x_{i+1}} \\ &= \lambda_k(S_a^*, S_b^*) + a_{p_i} - c_{x_i} - c_{x_{i+1}} \\ &\leq \lambda_k(S_a^*, S_b^*). \end{aligned}$$

For the case where $n^* = i$, we know that $|S_b^t| = m^* + 2$ and $S_b^t = S_b^* \cup \{s_{x_i}, s_{\beta_1}^t\}$. Thus, $s_{\beta_2}^t = \max(S_b^t, b, 1)$. Since there is an S_r -improvement on s_{p_i} at iteration t of Algorithm 2, $\Delta_r^t = b_{\beta_1}^t + b_{\beta_2}^t - a_{p_i} \geq b_{x_i} + b_{\beta_1}^t - a_{x_i}$, namely, $b_{\beta_2}^t - a_{p_i} \geq b_{x_i} - a_{x_i} = -c_{x_i}$. Therefore,

$$\begin{aligned} & \lambda_k(S_a^* + s_{p_i} - s_{x_i}, S_b^* + s_{x_i} - s_{\beta_2}^t) \\ &= \lambda_k(S_a^*, S_b^*) + a_{p_i} - a_{x_i} + b_{x_i} - b_{\beta_2}^t \\ &= \lambda_k(S_a^*, S_b^*) + a_{p_i} - c_{x_i} - b_{\beta_2}^t \\ &\leq \lambda_k(S_a^*, S_b^*). \end{aligned}$$

In the former case, $|S_a^*|$ is not minimum among all optimum solutions. In the latter case, the lexicographic order of S_a^* is not minimum among all optimum solutions with $|S_a^*| = n^*$. Therefore, this case is impossible.

Case 3: $\ell_{x_i} > \ell_{p_i}$ and $s_{p_i} = s_{\beta_1}^t$.

In this case, we can find that $s_{p_i} \notin S_a^*$ and $s_{p_i} \notin S_b^*$. The reason for $s_{p_i} \notin S_b^*$ is that every b -value in S_b^* is smaller than or equal to $s_{\beta_1}^t$. Then, by Lemmas 5, 6, and 7, $a_{x_j} < a_{p_i}$, $s_{x_j} \in S_b^t$, and $s_{x_j} \neq s_{\beta_1}^t$ for $j = i, i+1, \dots, n^*$. Since $b_{\beta_1}^t + b_{\beta_2}^t - a_{p_i} < b_{\beta_1}^t + b_{\beta_2}^t - a_{x_j}$ and s_{p_i} is the selected improving element at iteration t of Algorithm 2, none of s_{x_j} is $s_{\beta_2}^t$ for $j = i, i+1, \dots, n^*$. Then, by a similar reasoning as Case 2, we can find that $\lambda_k((S_a^* + s_{p_i}) \setminus \{s_{x_i}, s_{x_{i+1}}\}, S_b^* \cup \{s_{x_i}, s_{x_{i+1}}\}) \leq \lambda_k(S_a^*, S_b^*)$ when $n^* > i$ and $\lambda_k(S_a^* + s_{p_i} - s_{x_i}, S_b^* + s_{x_i} - s_{\beta_2}^t) \leq \lambda_k(S_a^*, S_b^*)$ when $n^* = i$. However, both cases are impossible.

Case 4: $\ell_{x_i} > \ell_{p_i}$ and $s_{p_i} \in S_b^t - s_{\beta_1}^t$.

Let us first consider the case where $s_{p_i} \notin S_b^*$. Then, by Lemmas 5, 6, and 7, $a_{x_j} < a_{p_i}$, $s_{x_j} \in S_b^t$, and $s_{x_j} \neq s_{\beta_1}^t$ for $j = i, \dots, n^*$. If $n^* = i$, then $s_{p_i} \in S_b^* = S_b^t \setminus \{s_{\beta_1}^t, s_{x_i}\}$. It contradicts the

assumption that $s_{p_i} \notin S_b^*$. Thus, $n^* > i$. Since s_{p_i} is in S_b^t , s_{p_i} is not a forbidden element at iteration t of Algorithm 2 and $(a_{x_i} - b_{x_i}) + (a_{x_{i+1}} - b_{x_{i+1}}) \geq a_{p_i}$. With a similar derivation in Case 2, we can find that $\lambda_k((S_a^* + s_{p_i}) \setminus \{s_{x_i}, s_{x_{i+1}}\}, S_b^* \cup \{s_{x_i}, s_{x_{i+1}}\}) \leq \lambda_k(S_a^*, S_b^*)$ which is a contradiction.

Now we consider the case where $s_{p_i} \in S_b^*$. Let $s_{x_j} = \min(S_i, a, 1)$ where $S_i = \{s_{x_i}, s_{x_{i+1}}, \dots, s_{x_n^*}\}$. There are three subcases needed to consider.

Subcase 4.1: s_{x_j} is a forbidden element at iteration t of Algorithm 2.

By definition, there exist two elements $s_u, s_v \in S_b^t$, such that $\max\{a_u, a_v\} < a_{x_j}$ and $(a_u - b_u) + (a_v - b_v) < a_{x_j}$. Since $s_{x_j} = \min(S_i, a, 1)$ and $\max\{a_u, a_v\} < a_{x_j}$, s_u and s_v are not in S_a^* . However, in this case, if $s_u \in S_b^*$ and $s_v \in S_b^*$, then

$$\begin{aligned} & \lambda_k((S_a^* \cup \{s_u, s_v\}) - s_{x_j}, S_b^* \setminus \{s_u, s_v\}) \\ &= \lambda_k(S_a^*, S_b^*) + a_u + a_v - a_{x_j} - b_u - b_v \\ &= \lambda_k(S_a^*, S_b^*) + (a_u - b_u) + (a_v - b_v) - a_{x_j} \\ &< \lambda_k(S_a^*, S_b^*). \end{aligned}$$

If at least one of s_u, s_v is not in S_b^* , say s_u , then $\lambda_k(S_a^* + s_u - s_{x_j}, S_b^*) < \lambda_k(S_a^*, S_b^*)$. It contradicts that $\lambda_k(S_a^*, S_b^*)$ is minimum.

Subcase 4.2: $s_{x_j} \in S_b^t - s_{\beta_1}^t$ is not a forbidden element at iteration t of Algorithm 2.

Since $\ell_{x_i} > \ell_{p_i}$, $b_{\beta_1}^t + b_{p_i} - a_{p_i} \geq b_{\beta_1}^t + b_{x_j} - a_{x_j}$ which implies $b_{p_i} - a_{p_i} \geq b_{x_j} - a_{x_j}$. Due to $s_{x_j} \in S_a^*$ and $s_{p_i} \in S_b^*$, we can obtain

$$\begin{aligned} & \lambda_k(S_a^* + s_{p_i} - s_{x_j}, S_b^* + s_{x_j} - s_{p_i}) \\ &= \lambda_k(S_a^*, S_b^*) + a_{p_i} - a_{x_j} + b_{x_j} - b_{p_i} \\ &= \lambda_k(S_a^*, S_b^*) - (b_{p_i} - a_{p_i}) + (b_{x_j} - a_{x_j}) \\ &\leq \lambda_k(S_a^*, S_b^*). \end{aligned}$$

It contradicts the assumption that the lexicographic order of S_a^* is minimum among all optimum solutions with $|S_a^*| = n^*$.

Subcase 4.3: $s_{x_j} \notin S_b^t - s_{\beta_1}^t$ is not a forbidden element at iteration t of Algorithm 2.

If $s_{x_j} \notin S_b^t - s_{\beta_1}^t$, then either $s_{x_j} = s_{\beta_1}^t$ or $s_{x_j} \in S_r^t$. In either case, $b_{\beta_1}^t + b_{\beta_2}^t - a_{x_j} \leq b_{\beta_1}^t + b_{p_i} - a_{p_i}$ since $\ell_{x_i} > \ell_{p_i}$. Since $S_b^* \subset S_b^t$, there exists an element, say s_e , which is in $S_b^t \setminus (S_a^* \cup S_b^*)$ with $b_e \leq b_{\beta_2}^t$. Therefore,

$$\begin{aligned} & \lambda_k(S_a^* + s_{p_i} - s_{x_j}, S_b^* + s_e - s_{p_i}) \\ &= \lambda_k(S_a^*, S_b^*) + a_{p_i} - a_{x_j} + b_e - b_{p_i} \\ &\leq \lambda_k(S_a^*, S_b^*) + a_{p_i} - a_{x_j} + b_{\beta_2}^t - b_{p_i} \\ &= \lambda_k(S_a^*, S_b^*) - (b_{p_i} - a_{p_i}) + (b_{\beta_2}^t - a_{x_j}) \\ &\leq \lambda_k(S_a^*, S_b^*). \end{aligned}$$

It also contradicts the assumption that the lexicographic order of S_a^* is minimum among all optimum solutions with $|S_a^*| = n^*$.

We conclude that $\ell_{x_i} = \ell_{p_i}$ for $1 \leq i \leq \min\{n^*, n'\}$. $\square$

Theorem 9 $S_a^* = S_a$ where S_a is obtained from *Algorithm 1*.

Proof.

By Lemma 8, we know that $\ell_{x_i} = \ell_{p_i}$ for $1 \leq i \leq \min\{n^*, n'\}$. All we have to prove is that $n^* = n'$. Hence, in the following, we prove that both $n^* < n'$ and $n^* > n'$ are impossible.

Case 1: $n^* < n'$.

Let $S_a^t = \{s_{p_1}, s_{p_2}, \dots, s_{p_{n^*}}\}$ and $S_b^t = \bigcup_{1 \leq i \leq 2(k-n^*)} \{Min(S \setminus S_a^t, b, i)\}$, where S_a^t and S_b^t are the two temporary sets S_a and S_b , respectively, in iteration t of *Algorithm 2*. It is clear that $S_b^t = S_b^*$. Since $n^* < n'$, S_{bc}^t or S_{rc}^t is not empty and $\ell_{p_{n^*+1}}$ must be an improving label. This implies that $a_{p_{n^*+1}} < b_{p_{n^*+1}} + b_{\beta_1}^t$ if $s_{p_{n^*+1}} \in S_b^t \setminus s_{\beta_1}^t$ and $a_{p_{n^*+1}} < b_{\beta_1}^t + b_{\beta_2}^t$ otherwise. Thus, in the former case, $\lambda_k(S_a^* + s_{p_{n^*+1}}, S_b^* \setminus \{s_{p_{n^*+1}}, s_{\beta_1}^t\}) < \lambda_k(S_a^*, S_b^*)$. Similarly, $\lambda_k(S_a^* + s_{p_{n^*+1}}, S_b^* \setminus \{s_{\beta_1}^t, s_{\beta_2}^t\}) < \lambda_k(S_a^*, S_b^*)$ for the latter case. In either case, it contradicts our assumption that $\lambda_k(S_a^*, S_b^*)$ is minimum.

Case 2: $n^* > n'$.

Let $S_a^t = \{s_{p_1}, s_{p_2}, \dots, s_{p_{n'}}\}$ and $S_b^t = \bigcup_{1 \leq i \leq 2(k-n')}\{Min(S \setminus S_a^t, b, i)\}$ where iteration t is the ending iteration of *Algorithm 2*. It is clear that $S_a^t = S_a$, $S_b^t = S_b$, and both S_{bc}^t and S_{rc}^t are empty. Therefore, either no candidate exists in $S_a^* - S_a^t$ or there is a forbidden candidate in $S_a^* - S_a^t$ with respect to S_a^t and S_b^t . If no candidate exists in $S_a^* - S_a^t$, then there exists no element, say s_x , in $S_a^* - S_a^t$ such that $\lambda_k(S_a^t + s_x, S_b^t \setminus \{s_{\beta_1}^t, s_x\}) < \lambda_k(S_a^t, S_b^t)$ if $s_x \in S_b^t - s_{\beta_1}^t$ or $\lambda_k(S_a^t + s_x, S_b^t \setminus \{s_{\beta_1}^t, s_{\beta_2}^t\}) < \lambda_k(S_a^t, S_b^t)$ otherwise. Thus, it is impossible that $n^* > n'$. Now we consider the case where there is a forbidden candidate with respect to S_b^t and S_r^t . By Lemma 2, there must exist a nonforbidden candidate in $S_a^* - S_a^t$ with respect to S_a^t and S_b^t . Then, by a similar argument as the previous case, it is also impossible. This completes the proof. $\square$

Corollary 10 $\lambda_k(S_a, S_b) = \lambda_k(S_a^*, S_b^*)$ where S_a and S_b are the output of *Algorithm 1*.

4 Detecting Forbidden Elements

In *Algorithm 1*, it is a time-consuming step, namely Step 2, to determine whether a candidate element is a forbidden element or not. If we solve this problem by checking exhaustively all possible pairs of elements in S_b , then it will take $O(k^2)$ time for each candidate in S . Thus, the total checking time will be $O(k^3)$. If we want to reduce the time-complexity of *Algorithm 1*, then we have to find an efficient algorithm for detecting whether a candidate is a forbidden element or not.

By using the algorithm in [3] for solving the range minima query problem, we can use $O(1)$ time to tell if a candidate is a forbidden element or not. Before describing the data structure for detecting a forbidden element, we introduce the range minima query problem briefly as follows.

Problem “RMQ” (The Range Minima Query problem).

We are given an array $\mathcal{C}[1..n]$ of numbers. We are asked to answer the following queries :

Query: For indices i and j between 1 and n , query $RMQ_{\mathcal{C}}(x, y)$ returns the index of the smallest element in the subarray $\mathcal{C}[i..j]$. (When the context is clear, we drop the subscript $\mathcal{C}$ on the RMQ.

Theorem 11 [3] After an $O(n)$ time preprocessing, each query of the RMQ problem can be answered in a constant time.

Corollary 12 Given an array $\mathcal{C}$ of n elements and indices i and j , the first k smallest values can be found in $O(k \log k)$ time after an $O(n)$ time preprocessing.

Proof. After we find the minimum value among the range $[i, j]$, the range will be split into two smaller subranges $[i, t-1]$ and $[t+1, j]$ where t is the index of the minimum value. Note that if $t = i$ or j , then there is only one subrange. By using a Fibonacci heap [1] to record the minimum values and their corresponding indices in $[i, t-1]$ and $[t+1, j]$ repeatedly, the first k smallest values can be found in $O(k \log k)$ time. $\square$

Now we describe our algorithm for finding nonforbidden elements.

Note that it is not necessary to sort the elements in S_b according to their a -values every time in order to obtain array $\mathcal{C}$ when *Algorithm 3* is invoked. We only need to remove the two related elements, which are s_x and s_{β_1} or s_{β_1} and s_{β_2} , from the previous array $\mathcal{C}$. Thus, Step 1 of *Algorithm 3* can be done in $O(k)$ time. By using the algorithm in

Algorithm 3**Input:** A set S_c contains all candidates of S .**Output:** S_{bc} and S_{rc} .**begin**

Step 1. Construct an array $\mathcal{C}$ in which index i corresponds to the element $\text{Min}\{S_b, a, i\}$ and $\mathcal{C}[i]$ contains its c -value. $S_{bc} = S_{rc} = \emptyset$.

Step 2. For each candidate $s_x \in S_c$,
if s_x is also in S_b , then do Substeps 2.1~2.4.

Substep 2.1. Let j be the index of s_x in array $\mathcal{C}$. Select the first three minimum c -values from $\mathcal{C}[1..j-1]$. Assume that they are elements s_{i_1} , s_{i_2} , and s_{i_3} .

Substep 2.2. Remove $\text{Max}(S_b - s_x, b, 1)$, if it exists, from the above selected elements. Assume without loss of generality that s_{i_1} and s_{i_2} are two remained elements.

Substep 2.3. If $c_{i_1} + c_{i_2} < a_x$, then
 $S_{bc} = S_{bc} \cup \{s_x\}$.

If s_x is s_δ , then do Substeps 2.4~2.6.

Substep 2.4. Let $j = |\mathcal{C}|$.
Select the first four minimum c -values from $\mathcal{C}[1..j]$. Assume that they are elements s_{i_1} , s_{i_2} , s_{i_3} and s_{i_4} .

Substep 2.5. Remove s_{β_1} and s_{β_2} , if exist, from the above selected elements. Assume without loss of generality that s_{i_1} and s_{i_2} are two remained elements.

Substep 2.6. If $c_{i_1} + c_{i_2} < a_x$, then
 $S_{rc} = S_{rc} \cup \{s_x\}$.

end

[3], Step 3 takes $O(1)$ time to determine whether s_x is forbidden or not. Thus, for detecting all candidates takes $O(k)$ time. Hence, the time-complexity of Algorithm 3 is $O(k)$. For clarity, by using the previous example, Figure 1 shows the data structure used in Algorithm 3. Figure 1(a) describes the data structure of S_b . The elements in S_b are sorted according to their a -values and b -values, respectively, and each sorted list is arranged as a doubly linked list. Pointer Pr_α points to the minimum value of the former sorted lists while Pr_{β_1} point to the maximum element of the latter sorted list. Thus, we can construct array $\mathcal{C}$ efficiently and update S_b in $O(1)$ time in each iteration. Figure 1(b) shows the elements in S_r which are maintained by a Fibonacci heap. Note that, in this example, it is a list in the beginning.

Now we summarize our result as the following theorem.

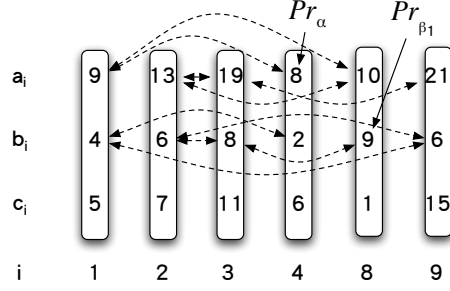
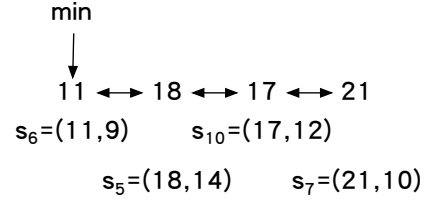
(a) the data structure of S_b (b) the data structure of S_r

Figure 1: The data structure used in Algorithm 1.

Theorem 13 Algorithm 1 solves the composition problem correctly in $O(k^2 + n \log n)$ time.

Proof. The correctness of Algorithm 1 is shown by Theorem 9 and Corollary 10. In the following, we shall analyze the time-complexity of Algorithm 1.

First, in Step 1 of Algorithm 1, we sort all elements in S according to their b -values. And, let $S_b = \bigcup_{1 \leq i \leq 2k} \{\text{Min}(S, b, i)\}$ and $S_r = S \setminus (S_a \cup S_b)$. Note that, in Step 1, S_b -candidates can also be found in $O(k)$ time. In addition, we also sort all elements in S_b according to their a -values which is used to construct array $\mathcal{C}$. Furthermore, the elements in S_r are maintained by a Fibonacci heap which is also created in Step 1. Note that, by Proposition 1, the Fibonacci heap contains at most k elements and takes $O(k)$ time to create. Hence, the total time of Step 1 is still $O(n \log n)$. In Step 2, it needs $O(k)$ time to build array $\mathcal{C}$ of the adjusted S_b in each iteration. By Corollary 12, the query time takes only $O(1)$ for checking whether a candidate element is forbidden or not. Thus, the total time taken by Step 2 is $O(k^2)$. In Step 3, it will spend $O(1)$ and $O(\log k)$ time to update S_b and S_r , respectively, in each iteration. Thus, k iterations will take $O(k^2)$ time. Therefore, the total time complexity of Algorithm 1 is $O(k^2 + n \log n)$. This completes the

proof. $\square$

5 Concluding Remarks

In this paper, we define a new scheduling problem called the composition problem. We also propose an $O(k^2 + n \log n)$ time algorithm for solving this problem. A more general version of our problem can be defined as follows. Given $S = \{(a_i, b_i) | 1 \leq i \leq n\}$, how to find two disjoint subsets S_a and S_b from S with $|S_a| + \frac{|S_b|}{p} = k$ such that $\sum_{s_i \in S_a} a_i + \sum_{s_i \in S_b} b_i$ is minimum where p and k are two given integers and both a_i and b_i are real numbers with $a_i > b_i$? Since the general version has two parameters p and k , we can call it the p/k -composition problem. Actually, **Algorithm 3** is prepared for solving this general version. As a consequence, the composition problem solved in this paper is a $2/k$ -composition problem. The meaning behind the p/k -composition problem is that one element in a feasible solution set S_a can be replaced by p elements not in S_b if the replacement can get a better result. Following this concept, an even more general version of the p/k -composition problem will be the $q/p/k$ -composition problem in which we can replace q elements in S_a by p elements not in S_b . Therefore, our current composition problem is the $1/2/k$ -composition problem. We are trying to solve these problems now.

References

- [1] M. L. Fredman and R. E. Tarjan, Fibonacci heaps and their uses in improved network optimization algorithms, *Journal of the ACM*, vol. 34, 1987, pp. 596–615.
- [2] E. Horowitz and S. Sahni, *Fundamentals of Computer Algorithms*. Computer Science Press 1978.
- [3] M. A. Bender and M. Farach-Colton, The LCA problem revisited, *LATIN 2000 : Theoretical Informatics, 4th Latin American Symposium, Punta del Este, Uruguay*, 2000, pp. 88–94.